

RecoverGraph: Learning from Outcome-Verified Recovery Experience in LLM Coding Agents

Maha Khan*
mmahakhan24@gmail.com
Independent Researcher
Karachi, Sindh, Pakistan

Nabiha Waseem*
nabihawaseem03@gmail.com
Independent Researcher
Karachi, Sindh, Pakistan

Abstract

LLM coding agents can recover from execution failures within a task yet rediscover the same operational problems in later independent executions. We present *RecoverGraph*, a harness-side, non-parametric recovery-memory mechanism that converts observed failures, concrete recoveries, and objective execution outcomes into persistent, outcome-verified experience. During later tasks, RecoverGraph retrieves semantically relevant recovery patterns and applies deterministic chronology, verification, and applicability checks before selectively exposing guidance to the agent.

We evaluate RecoverGraph in four executable software-engineering case studies. Across three paired cases, task success was 2/3 under both baseline and treatment conditions; among two defensible recurrence opportunities, baseline repeated one previously observed failure whereas treatment repeated none. In one treatment case, the treatment trajectory used 35.0% fewer model requests, 42.1% fewer tool calls, 37.1% fewer total tokens, and 28.9% less wall-clock time. Another case exhibited abstention when no historical recovery satisfied the applicability criteria, while two cases retrieved the intended prior recovery only after the agent had already made the key corrective change. These observations suggest that outcome-verified experience can be selectively reused across agent executions, while also showing that successful retrieval alone is insufficient: effective experience reuse depends on selectivity and on introducing relevant guidance at a behaviorally useful point in the agent trajectory.

Keywords

LLM agents, coding agents, agent memory, experience reuse, test-time adaptation, failure recovery, software engineering

1 Introduction

Large language model (LLM) agents increasingly perform software-engineering tasks through iterative reasoning and tool use. During execution, an agent may encounter operational failures such as invalid command usage, dependency incompatibilities, configuration errors, or environment-specific constraints. Although the agent may recover within that execution, the resulting experience is typically unavailable to later independent runs. A subsequent task can therefore rediscover a failure for which an earlier execution already observed a successful recovery.

Prior work has shown that LLM agents can reuse previous experience at test time without updating model weights. Reflexion stores verbal reflections derived from task feedback in episodic memory [7]; Agent Workflow Memory induces reusable workflows from earlier tasks [9]; and ReasoningBank distills strategies from

successful and failed trajectories for later retrieval [5]. Recent evaluation work also emphasizes whether accumulated memory changes later agent behavior rather than only supporting static recall [1]. These directions motivate a narrower question: can objective execution feedback be converted into persistent experience that an LLM coding agent can selectively reuse during later related tasks?

We investigate this question through *RecoverGraph*, a harness-side mechanism for non-parametric, outcome-conditioned adaptation. RecoverGraph represents an observed recovery as a structured *RecoveryPattern* derived from a failed action, the concrete recovery that followed it, and an objectively evaluated outcome. The pattern preserves provenance to the source failure, task, resolution, outcome, and execution environment. During a later execution, semantically relevant historical patterns are retrieved and then subjected to deterministic chronology, verification, and applicability checks before any recovery guidance is exposed to the coding agent.

RecoverGraph does not update the underlying language-model parameters and is therefore not a conventional reinforcement-learning algorithm. Instead, objective execution outcomes act as feedback for constructing reusable non-parametric experience. This experience persists across executions and can be selectively introduced into the context of a later task, allowing the agent to reconsider its behavior without policy optimization or fine-tuning.

Our evaluation uses four case-study experiments. Together, they examine three distinct aspects of cross-episode experience reuse: a favorable treatment trajectory in which a previously observed failure does not recur, abstention when no defensible recovery is applicable, and two cases in which the intended historical recovery is retrieved only after the agent has already made the key corrective change. These cases motivate a distinction between experience availability, experience selection, and intervention timing.

We therefore ask:

RQ: Can an LLM coding agent reuse outcome-verified experience from earlier executions to adapt its behavior on related software-engineering tasks, and how do retrieval selectivity and intervention timing affect that reuse?

This work makes four contributions. First, we introduce outcome-verified recovery memory that converts observed failures, recoveries, and objective execution outcomes into persistent reusable experience. Second, we provide a selective cross-episode retrieval mechanism that combines semantic candidate retrieval with deterministic chronology, verification, and applicability checks, including the ability to abstain when transfer is not justified. Third, we report four case-study experiments covering a favorable treatment trajectory, conservative abstention, and two correct-retrieval but

*Both authors contributed equally to this research.

late-intervention cases. Fourth, the experiments expose intervention timing as a practical constraint on experience reuse: retrieving relevant prior experience is insufficient if that experience enters the agent trajectory too late to influence subsequent behavior.

Although we evaluate RecoverGraph in executable software-engineering tasks, the same selective experience-reuse problem can arise in tool-using financial workflows, where prior operational experience must be reused cautiously while preserving provenance, auditability, and the ability to abstain from inappropriate transfer.

2 RecoverGraph

RecoverGraph is a harness-side recovery-memory mechanism for LLM coding agents. It converts historical failure–recovery–outcome evidence into reusable *RecoveryPatterns* and selectively exposes applicable experience as guidance before later tool actions execute. The mechanism does not modify the language-model parameters and remains advisory: it neither rewrites planned actions nor executes historical recoveries autonomously. Instead, it retrieves relevant prior experience, checks whether that experience is chronologically valid and applicable to the current execution, and gives the coding agent an opportunity to reconsider its planned action. **Agent integration.** The coding agent is implemented with PydanticAI and interacts with its workspace through five controlled tools: `read_file`, `write_file`, `edit_file`, `run_tests`, and `run_command`. RecoverGraph is integrated at the pre-tool action boundary: in treatment runs, applicable recovery guidance is returned to the model before the planned tool operation executes, allowing the agent to reconsider or retain its action. RecoverGraph itself neither modifies the workspace nor executes a stored recovery.

Figure 1 summarizes the acquisition and cross-episode experience-reuse pipeline.

2.1 Outcome-Verified Recovery Memory

RecoverGraph stores a structured abstraction of recovery evidence rather than replaying an entire historical trajectory. A *FailureEpisode* records an operational failure observed during an earlier execution. A subsequent concrete change associated with recovery is represented as a *Resolution*, and the resulting task state is evaluated through an external *Outcome*. Eligible failure–resolution–outcome lineages are distilled into reusable *RecoveryPatterns*.

A *RecoveryPattern* contains concise recovery guidance together with provenance to the source failure, resolution, outcome, task, and environment. It also retains information required for later applicability checks, including the originating tool or operation, chronological position, verification state, and environment constraints. A later execution can therefore reuse an operational lesson without replaying the complete source trajectory.

Here, *outcome-verified* does not mean causally verified. RecoverGraph does not interpret *failure* $\rightarrow$ *change* $\rightarrow$ *success* as proof that the observed change caused the successful outcome. The stored pattern instead represents an observed recovery associated with an objectively evaluated outcome. Its relevance to a later action is evaluated separately before guidance is issued.

2.2 Acquisition and Recovery Abstraction

Trusted failure–recovery–outcome evidence from the acquisition stage is distilled into structured *RecoveryPatterns* using `cohere/north-mini-code:free`. For each eligible execution, RecoverGraph constructs a bounded evidence package containing the task context, failed action, failure characteristics, concrete recovery, objective outcome, and relevant environment metadata.

The abstraction model generates the semantic content of the *RecoveryPattern*, including a concise description and actionable recovery guidance. Provenance, chronology, verification state, and other deterministic metadata are attached by the surrounding system rather than inferred solely by the abstraction model.

The acquisition corpus contains five frozen *RecoveryPatterns*. The corpus is frozen before transfer evaluation and remains read-only during the reported experiments. Transfer executions may retrieve from this historical experience but cannot add or modify *RecoveryPatterns*. The study therefore evaluates cross-episode reuse of fixed prior experience rather than continual online memory construction.

2.3 Selective Retrieval and Pre-Execution Advice

Transfer executions use `qwen/qwen3-coder` as the coding agent. *RecoveryPatterns* are embedded using `all-MiniLM-L6-v2` via Hugging Face Inference, based on MiniLM and Sentence-Transformer representations [6, 8]. The resulting 384-dimensional normalized vectors are stored in Neo4j and used for semantic candidate retrieval.

For a later execution, the retrieval query is constructed only from information available at that point: the task description, planned action and tool or operation, current environment, and chronological position. Semantic similarity generates candidate *RecoveryPatterns* but does not by itself determine whether guidance is shown.

Candidates subsequently pass deterministic checks for strict chronology, verification status, invalidation or staleness, tool and operation compatibility, and environment or version applicability. Benchmark family labels, expected recoveries, gold patches, recurrence annotations, evaluator metadata, and future-task information are excluded from the retrieval path.

If an applicable *RecoveryPattern* remains, RecoverGraph exposes concise recovery guidance at the pre-tool boundary before the selected operation executes. The coding agent may then reconsider and either revise or retain its planned action. If no pattern passes the eligibility checks, RecoverGraph abstains from providing recovery guidance. This separation between experience retrieval and selective intervention is central to the reported experiments, where relevant experience may be retrieved, withheld, or delivered too late to influence subsequent agent behavior.

3 Experimental Design

We evaluate RecoverGraph through four case-study experiments. A five-pattern recovery corpus, distilled from five earlier software-engineering executions, is frozen before evaluation and remains read-only throughout the reported transfer runs. The coding agent does not update its model parameters between executions. The evaluation therefore examines whether fixed, outcome-verified

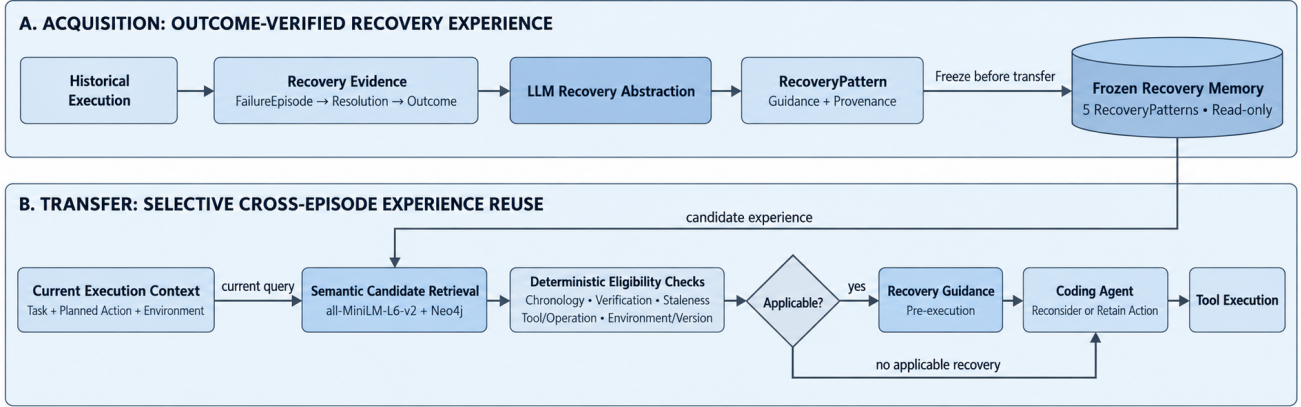


Figure 1: RecoverGraph acquisition and cross-episode experience-reuse pipeline. Earlier executions provide failure–recovery–outcome evidence that is distilled into RecoveryPatterns and frozen before transfer. During a later execution, semantic retrieval generates candidate experience, while deterministic eligibility checks determine whether recovery guidance is exposed before tool execution.

Table 1: Repository environments used in the reported cases.

Case	Repository	Revision
EXP1	Tornado	d5ac65c1
EXP2	Jinja	ada0a9a6
EXP3	SQLGlott	036601ba
EXP4	Rich	9d8f9a37

experience is retrieved, withheld, or introduced into later agent trajectories.

3.1 Cases and Experimental Conditions

The study uses executable software-engineering environments derived from SWE-smith [12]. RecoverGraph defines its own manually validated recurrence relationships and acquisition–transfer structure on top of these environments; the reported cases are not an official SWE-smith benchmark split.

Table 1 summarizes the four reported repository environments. Short revision identifiers are shown where available; exact container identifiers and recorded environment metadata are preserved with the reproducibility artifacts.

Each case is evaluated under two conditions:

- **B0:** the coding agent operates without persistent RecoverGraph recovery memory.
- **T:** the same coding agent may retrieve guidance from the frozen five-pattern RecoveryPattern corpus.

All reported runs use the Kilo provider with qwen/qwen3-coder and the same frozen system prompt. Runs are limited to 24 model requests and 28 tool actions, with a 300 s task timeout, 30 s command

timeout, and 120 s test timeout. Each run executes in an isolated task workspace associated with its SWE-smith environment. Within each B0/T comparison, the task objective, repository environment, coding model, prompt, tools, resource limits, and objective evaluator are held fixed; the treatment condition additionally enables the frozen RecoverGraph advisory mechanism.

The reported experiments are case studies rather than statistically independent confirmatory trials. Aggregate task-success comparisons are reported over EXP1–EXP3, while EXP4 is analyzed as an additional retrieval-and-intervention-timing case.

3.2 Metrics and Evaluation

The primary recurrence measure is *Repeated Failure Rate* (RFR), defined as the number of previously observed failures that recur divided by the number of manually defensible opportunities to repeat a known failure. We report the raw numerator and denominator together with the rate.

False Advice Rate (FAR) was defined as the proportion of advice events that are incorrect or inapplicable. However, the retained artifacts do not contain event-level objective adjudication sufficient to compute a defensible FAR estimate, so we do not report one. Task success is determined using the frozen SWE-smith FAIL_TO_PASS tests executed with pytest. We additionally inspect whether RecoverGraph retrieves the intended historical recovery, whether guidance is issued or withheld, when guidance enters the agent trajectory relative to the key corrective change, and whether subsequent agent behavior changes. Where available, we also report descriptive efficiency measures including model requests, tool calls, token usage, and wall-clock time.

Table 2: EXP1 baseline and RecoverGraph treatment results.

Metric	B0	T	Change
Task success	Yes	Yes	–
Known-failure recurrence	1	0	Avoided
Model requests	20	13	–35.0%
Tool calls	19	11	–42.1%
Total tokens	182,978	115,102	–37.1%
Wall-clock time (s)	101.1	71.9	–28.9%

Because the reported evidence consists of a small development case-study set, we report descriptive task-level comparisons rather than inferential statistical claims.

Artifacts. Source code, experiment configurations, and reproducibility artifacts are available at https://github.com/mahakhan-mk/Recover_Graph.

4 Results

We report a focused four-case evaluation of RecoverGraph under baseline (B0) and treatment (T) conditions.

Across the cases with defensible opportunities for recurrence of a previously observed failure, B0 repeated a known failure in 1/2 opportunities (50%), whereas T repeated 0/2 (0%). The resulting 50 percentage-point difference is descriptive only; the small sample does not support inferential claims. Across EXP1–EXP3, task success was 2/3 under both B0 and T, indicating that the observed difference was specific to repeated-failure behavior rather than overall task completion.

EXP1 provides the clearest evidence of changed treatment behavior. Both B0 and T completed the task successfully, but the known failure recurred under B0 and did not recur under T. RecoverGraph retrieved the intended historical recovery, and the run recorded a subsequent action change after advice. Table 2 summarizes the EXP1 trajectory. The treatment used fewer model requests, tool calls, tokens, and wall-clock time than B0. These efficiency differences are descriptive and should not be interpreted as causal effects of RecoverGraph advice.

EXP2 illustrates selective abstention: although the frozen recovery memory was available, no historical recovery satisfied the applicability criteria, so RecoverGraph withheld recovery guidance. Both B0 and T failed. This case highlights the role of applicability filtering in preventing unsupported cross-episode transfer.

In EXP3 and EXP4, RecoverGraph retrieved the intended historical recovery, but the relevant guidance entered the agent trajectory only after the agent had already made the key corrective change. These cases show that successful experience retrieval alone is not sufficient for effective reuse in LLM coding agents. The timing of memory delivery is itself an important design variable: historical experience must become available early enough in the decision process to influence subsequent agent behavior.

Taken together, the four cases illustrate three distinct properties of cross-episode experience reuse. EXP1 provides a favorable treatment trajectory in which a previously observed failure recurred under B0 but not under treatment. EXP2 illustrates selective abstention when no historical recovery satisfied the applicability criteria.

Table 3: Summary of task outcomes, recurrence observations, and treatment advice across the four RecoverGraph cases.

Case	Success B0/T	Recur. B0/T	T advice	Interpretation
EXP1	Y/Y	1/0	1	Recurrence avoided under T
EXP2	N/N	–	0	No applicable advice
EXP3	Y/Y	0/0	1	Correct retrieval, late
EXP4	Y/Y	–	1	Correct retrieval, late

EXP3 and EXP4 show that identifying relevant prior experience and introducing it at a behaviorally useful point are distinct stages. These observations motivate evaluating agent memory not only by retrieval quality, but also by transfer selectivity, intervention timing, and subsequent behavioral effects. Table 3 summarizes task outcomes, recurrence observations, and treatment advice across the four reported cases.

5 Related Work

Experience reuse in LLM agents. Several approaches allow agents to reuse prior experience at test time without updating model weights. Reflexion stores verbal reflections derived from task feedback in episodic memory to guide later trials [7]. Agent Workflow Memory (AWM) induces reusable workflows from previous tasks and retrieves them to guide subsequent actions [9]. More recently, ReasoningBank distills reusable reasoning strategies from both successful and failed trajectories and retrieves them during later tasks [5]. RecoverGraph builds on this broader experience-reuse paradigm but treats the failure–recovery–outcome lineage as the unit of reusable experience. Each pattern preserves an observed failure, the concrete recovery that followed it, and an objectively evaluated outcome, while later reuse is gated by chronology, verification, and applicability checks rather than model-parameter updates.

Structured and graph-based memory. Graph structure itself is not the contribution of RecoverGraph. A-MEM organizes agent memories as dynamically linked and evolving memory networks [10], while MAGMA represents memory across semantic, temporal, causal, and entity graphs and performs query-adaptive retrieval over these views [3]. These systems establish graph-structured and relational agent memory as existing research directions. RecoverGraph instead uses structured memory to preserve recovery provenance and support deterministic chronology, verification, and applicability checks around operational RecoveryPatterns. The graph therefore serves as an implementation substrate for selective experience reuse rather than as the primary novelty claim.

Memory evaluation and behavioral reuse. Recent benchmarks increasingly evaluate whether accumulated experience affects later behavior rather than measuring retrieval in isolation. MemoryAgentBench studies memory through incremental multi-turn interaction and test-time learning [2], while MemoryArena evaluates interdependent multi-session tasks in which experience acquired in earlier subtasks must guide later actions [1]. RecoverGraph adopts this behavioral view of memory evaluation but focuses on outcome-verified recovery experience in executable software-engineering tasks. In particular, our evaluation separates

whether relevant experience is retrieved, whether transfer is withheld when no applicable recovery exists, and whether retrieved experience enters the agent trajectory early enough to influence subsequent behavior.

6 Conclusion and Future Work

RecoverGraph provides a harness-side mechanism for reusing outcome-verified recovery experience across independent LLM-agent executions without updating model parameters. Across the reported case studies, EXP1 showed a favorable treatment trajectory in which a previously observed failure recurred under B0 but not under treatment; EXP2 illustrated selective abstention when no stored recovery satisfied the applicability criteria; and EXP3–EXP4 showed that relevant prior experience can be retrieved yet arrive too late to influence subsequent agent behavior. Together, these cases suggest that effective cross-episode experience reuse depends not only on retrieval quality, but also on transfer selectivity and intervention timing.

The study is limited by its small case-study sample, software-engineering setting, fixed five-pattern recovery corpus, and the absence of event-level adjudication required to compute False Advice Rate. The results are therefore descriptive rather than evidence of broad generalization. Future work should investigate earlier and more adaptive intervention strategies and evaluate RecoverGraph in other tool-using domains. Financial-agent workflows are one promising setting because they combine tool-using automation with complex, changing operational processes [11]. Corporate-action processing is a particularly relevant example: event information such as dividends, splits, and other security updates may pass through multiple intermediaries and data sources, creating substantial reconciliation and operational-processing challenges [4]. In such settings, outcome-verified recovery memory could be evaluated as an operational reliability mechanism that selectively reuses prior experience while abstaining when the current context is not sufficiently comparable to the historical case.

References

- [1] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. 2026. Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. PMLR, 41975–42005. <https://proceedings.mlr.press/v306/he26am.html>
- [2] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2026. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *The Fourteenth International Conference on Learning Representations*.
- [3] Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026. MAGMA: A Multi-Graph based Agentic Memory Architecture for AI Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 36848–36865. doi:10.18653/v1/2026.acl-long.1709
- [4] Timothy Lind. 2025. Revolutionising the Power of Corporate Actions Data. DTCC Insights.
- [5] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=jL7fwchScm>
- [6] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 3982–3992. doi:10.18653/v1/D19-1410
- [7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [8] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33.
- [9] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. <https://proceedings.mlr.press/v267/wang25bx.html>
- [10] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-0593
- [11] Hongyang Yang, Likun Lin, Yang She, Xinyu Liao, Jiaoyang Wang, Runjia Zhang, Yuquan Mo, and Christina Dan Wang. 2025. FinRobot: Generative Business Process AI Agents for Enterprise Resource Planning in Finance. *CoRR* abs/2506.01423 (2025). doi:10.48550/arXiv.2506.01423
- [12] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling Data for Software Engineering Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-3239